

Phase Zero

Un autre challenge de la catégorie reverse fournissait un exécutable linux nommé `bs_phase_zero`.

Résolution du challenge

On ouvre le fichier dans Ghidra. Le main lit ce qui est écrit dans le terminal, appelle une fonction `bs_guard` et affiche un message de refus ou non selon la valeur retournée.

```
bVar1 = bs_guard(local_28);
if ((int)CONCAT71(extraout_var,bVar1) == 0) {
    puts("Breizh Systems :: status: denied");
    uVar4 = 1;
}
else {
    puts("Breizh Systems :: status: ok");
    uVar4 = 0;
}
,
```

En analysant la fonction `bs_guard`, on s'aperçoit qu'elle appelle aussi une autre fonction appelée `bs_xor_decode` pour comparer l'une des valeurs modifiées par la fonction à l'input. Celle-ci est beaucoup plus intéressante.

```
void bs_xor_decode(long param_1,long param_2)
{
    ulong uVar1;

    uVar1 = 0;
    do {
        *(byte *) (param_1 + uVar1) = (&DAT_001020fd)[uVar1 % 6] ^ *(byte *) (param_2 + uVar1);
        uVar1 = uVar1 + 1;
    } while (uVar1 != 0x1d);
    return;
}
```

`bs_xor_decode` XORE le message situé à 0x1020e0 avec la clé située à 0x1020fd mod 6. Clé = 42 5A 48 43 54 46 = "BZHCTF"

Un petit programme python permet de retrouver la valeur originale et donne le flag `BZHCTF{15_7h15_7h3_r341_m41n}`. Malheureusement, ce n'est pas le bon... Il faut chercher ailleurs.

```

ctf > ctf1.py > ...
1
2 data = bytes([0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x39, 0x6b, 0x7d, 0x1c,
3             0x63, 0x2e, 0x73, 0x6f, 0x17, 0x74, 0x3c, 0x75, 0x1d, 0x28,
4             0x7b, 0x77, 0x65, 0x19, 0x2f, 0x6e, 0x79, 0x2d, 0x29
5         ])
6
7 key = b'BZHCTF'
8
9
10 decoded = bytes(data[i] ^ key[i % len(key)] for i in range(len(data)))
11 print(decoded.decode())

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 3

```

flupinette@MSI:~/4A-TP/rootme$ python ./ctf/ctf1.py
BZHCTF{15_7h15_7h3_r341_m41n}
flupinette@MSI:~/4A-TP/rootme$ ./ctf/bs_phase_zero
Breizh Systems :: artifact phase-zero
Breizh Systems :: enter token: BZHCTF{15_7h15_7h3_r341_m41n}
Breizh Systems :: status: denied

```

On retourne lire les fonctions. Visiblement quelque chose a été manqué, ce qui amène à aller regarder du côté des fonctions d'initialisation.

Elle a à peu près la même tête que le main mais appelle des fonctions différentes.

```

sVar3 = strchrspn(local_28, "\n");
local_28[sVar3] = '\0';
bVar1 = FUN_001012fe(local_28);
if ((int)CONCAT71(extraout_var, bVar1) != 0) {
    puts("Breizh Systems :: status: ok");
    /* WARNING: Subroutine does not return */
    exit(0);
}
puts("Breizh Systems :: status: denied");
/* WARNING: Subroutine does not return */
exit(1);
}

```

En avançant un peu, on tombe sur la fonction `FUN_00101281` qui construit le flag ainsi :

Les données de base sont à l'adresse `0x102110` : `66 6f 62 67 7d 6c 4f 1a 44 19 1c 75 1c 57 58 1c 52 57`. La clé est à l'adresse `0x102103` : `[02, 01, 00]` qui se répète grâce au modulo. La fonction construit le flag en faisant `flag[i] = data[i] + key[i%3]` puis `memfrob` XORe chaque octets avec 42.

```

void FUN_00101281(void *param_1)
{
    undefined1 auVar1 [16];
    ulong uVar2;

    uVar2 = 0;
    do {
        auVar1._8_8_ = 0;
        auVar1._0_8_ = uVar2;
        *(undefined1 *) ((long)param_1 + uVar2) =
            (&DAT_00102110)[uVar2] +
            (&DAT_00102103)
            [uVar2 - ((SUB168(auVar1 * ZEXT816(0xaaaaaaaaaaaaaab),8) & 0xfffffffffffffffe) + uVar2 / 3
                )];
        uVar2 = uVar2 + 1;
    } while (uVar2 != 0x12);
    memfrob(param_1,0x12);
    return;
}

```

Un programme python est codé pour retrouver le flag original et celui-là passe ! Le flag était BZHCTF{1n17_4rr4y}.

```

ctf > ctf2.py > ...
1  data = bytes([0x66, 0x6f, 0x62, 0x67, 0x7d, 0x6c, 0x4f, 0x1a, 0x44,
2      0x19, 0x1c, 0x75, 0x1c, 0x57, 0x58, 0x1c, 0x52, 0x57])
3
4  key = [0x02, 0x01, 0x00]
5
6  step1 = bytes((data[i] + key[i % 3]) & 0xFF for i in range(len(data)))
7
8  # memfrob
9  decoded = bytes(b ^ 0x2A for b in step1)
10
11 print(decoded.decode())
12

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 3

```

flupinette@MSI:~/4A-TP/rootme$ python ./ctf/ctf2.py
BZHCTF{1n17_4rr4y}
flupinette@MSI:~/4A-TP/rootme$ ./ctf/bs_phase_zero
Breizh Systems :: artifact phase-zero
Breizh Systems :: enter token: BZHCTF{1n17_4rr4y}
Breizh Systems :: status: ok

```